

A continuous rung

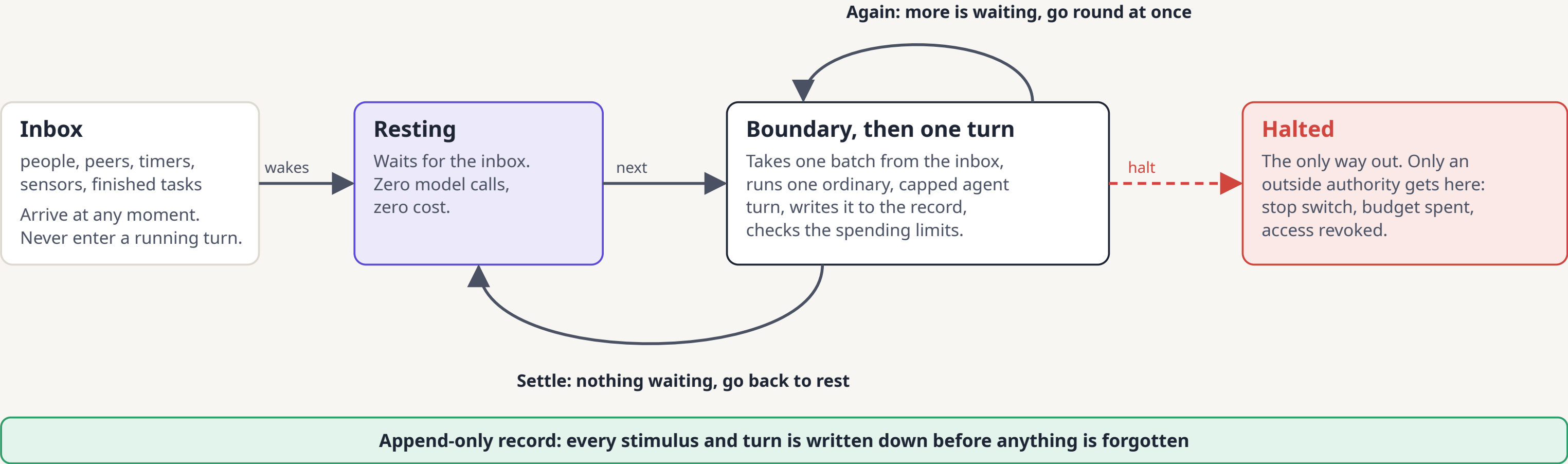
Can a rung agent run without ever reaching a natural end, taking in the world between turns? This is the short answer, the one thing to decide first, and seven questions for the owner.

1. **"Never exit" does not mean one endless model call.** It means turns that never reach a natural terminal, with carry between them and a boundary where the world gets in. The research this draws on says the same: continuity is a tick, not an analog process.
2. **Recommendation: add a continuous ladder in the product crate, not the kernel.** It rests for free, admits one batch of stimuli at each turn boundary, and can only be stopped by an outside authority. The existing agent turn is reused unchanged.
3. **The hard part is the record, not the loop.** Rung replays the whole session every turn, so an endless session fails in the limit. It needs a durable log, bounded context and spending limits.
4. **Smallest first slice costs \$0.** A flag on the agent, a fake stimulus source, a mock model, a log and a governor, with four pass or fail gates. Nothing has been built yet.

THE IDEA

The loop never ends, but each turn does

One new ladder wraps the existing agent turn. It adds a place to rest, a place where the world is let in, and one exit that the model cannot take.



Where rung stands now, and what each claim rests on

Every statement in the report carries a mark. Read the marks before relying on a claim.

VERIFIED read in the code

- An agent run is one bounded turn per prompt. Prompts already queue in one first-in, first-out line, so a stimulus that arrives mid-turn already waits for the turn to finish.
- That line takes one prompt per turn, with no merging, no priority, no rest state and no carry.
- The whole session is replayed on every turn. When context overflows, old tool results are trimmed in memory only, and the trim is not saved.
- A cancel flag is already checked before each model call and around each tool. A model call itself cannot be cancelled once started.

CLAIMED from the research records

- Results come from a related research project's frozen records, on small fixtures and an untrained model. Examples: memory re-entered once, as one labelled block, works; re-entered over and over, it destabilises the stream; a summary that replaces its predecessor overwrites its own notes.

INFERRED my reading, not tested

- That those findings carry over to hosted language models. They were measured on a different kind of model.
- The cost figures. \$21.60 a day at one idle turn a minute is an illustration, not a measurement.
- The size of the work, about 1,500 to 2,500 lines.
- That an endless session needs compaction by construction: user lines and assistant text are never trimmed, so it overflows on text alone.

Not done

- No rung build was started, nothing was run against a live model, and nothing was published.
- One correction to the original brief: the research's "codebook" experiment never ran, and its third experiment has not started.

Start with the smallest thing that proves the loop, at no cost

Everything lives in the product crate behind a flag, with the agent loop unchanged and a mock model standing in for a real one.

What gets built

- A **stimulus and inbox** with a directory source that takes message files.
- The **continuous ladder**: rest with zero calls, then admit, turn and record.
- Default **admission rules**: priority, merging of repeated readings, a batch size, class limits.
- A **bounded context**: a pinned start, a ring of the last few turns, the admitted batch. The log is written before anything leaves the ring.
- An **append-only log** and a **governor** with call and token limits and a stop file.
- A **scripted fake source**: operator messages, a burst of 20 peer messages, a once-a-second sensor, timers and one mid-turn interrupt.

Left for later slices: a cumulative carried note, a scheduled wake tool, the protocol extensions for hosted use, and checking each turn before its content is kept.

Four gates, fixed before the first run

MEASURE	PASS WHEN
Model calls during ten quiet minutes	exactly 0
Context size over 1,000 stimuli	flat once the ring fills, never above budget
Time from arrival to turn start	within one turn's duration, at p95; an interrupt within the call in flight
Hard kill and restart	every stimulus has exactly one outcome: admitted, merged, dropped with a reason, or pending. The gap is logged

Also checked: the spend cap reaches rest with no further calls, and the stop file halts within one call. Cost: \$0. One optional live smoke test under a \$1 cap, only if wanted.

Seven questions: authority, idleness, spending and boundary

Each has a recommendation. The ones marked shape the first slice; the rest can wait.

1. Authority SHAPES SLICE 1

Should rung ship a self-driven mode, where it owns timers and sources, or only boundary semantics for a caller who owns every loop?

Recommendation. Self-driven for slice 1, because the gates need rung to own an inbox. Build the same ladder so a caller can later own admission instead.

What the answer changes. Whether rung owns timers and sources at all, which decides slice 1's scope and the size of the work.

2. Idle policy

Is resting the only default, with a scheduled-wake tool allowed later? If so, what floor and what per-day count?

Recommendation. Rest only, with no idle turns. The wake tool comes after slice 1, with a floor of at least 5 minutes and a small per-day count.

What the answer changes. The self-turn guard, the cost model, and whether any turn can start without an outside stimulus.

3. Spending limits SHAPES SLICE 1

What token and dollar caps apply per hour and per day on hosted models, and who may refill them?

Recommendation. Conservative caps, with a soft stop that rests and a higher hard cap that halts. Only an operator or control message refills. The numbers are the owner's to set.

What the answer changes. The governor's defaults, and when the loop rests versus halts.

4. Boundary

Admit strictly at the next turn, as the brief said, or also steer at step boundaries for operator messages?

Recommendation. Strictly the next turn. An operator can already cancel a running turn. Step steering needs a kernel change and can be decided on its own later.

What the answer changes. Whether the inbox must reach inside the agent's carry, which weakens the guarantee that only cancel crosses into a turn.

Three more: first user, the research link and the protocol

Only the first user touches slice 1. The other two can be deferred without blocking it.

5. First user SHAPES SLICE 1

Which instance is the target: a standalone supervisor agent, the dormant rung identity on the task queue, or the research project's slow loop?

Recommendation. The standalone supervisor. It already works from a directory of message files, which is the shape slice 1's source takes.

What the answer changes. The fake source's shape, the inbox format, and which real workload proves the loop after the gates pass.

6. The research project

Should rung become a specialist the research project's loop calls on, and is GPU time wanted for three proposed experiments?

Recommendation. Not yet. That path needs an outcome of the research that has not been built, and its own doctrine says rung must be a specialist, never a conductor. Schedule the experiments separately.

What the answer changes. Nothing in slice 1. It decides only whether rung is shaped by a second, unrelated consumer.

7. Protocol

May one merged turn answer several prompt requests from the agent protocol, or must each prompt keep its own turn?

Recommendation. Keep one turn per prompt for now. The protocol defines one response per prompt, so merging needs a protocol decision, and a consumer that already merges above rung does not need it.

What the answer changes. How hosted use is wired: merged answers, a drain prompt, or a plain prompt per stimulus. Slice 1 uses no protocol.

If these are answered

Questions 1, 3 and 5 are enough to promote the first slice. The held backlog task for the first slice stays held until then.